INCIDENT RECOVERY REPORT

MEND AI

A reasoning engine: forms hypotheses from logs, metrics and diagnostics, fixes the most likely cause, re-checks; runs offline, designed knowing all 12 fault types, never tuned on held-out runs.

10 incidents · run `reasoner-dev` · generated 2026-10-30 06:51 UTC

100%	–	99	–
recovered, dev incidents	recovered, never-seen incidents	mean score, dev	mean score, never-seen

HOW TO READ THIS REPORT

Each incident was injected into a live five-service shop. The agent investigated and acted only through a guarded toolbox, then filed a root-cause report. An independent verifier sent real user traffic over a soak window and checked that each fault's root cause was actually gone. Score: 60 points for a verified recovery, 30 for naming the right root cause, 10 for touching nothing that was not broken. Each incident page ends with recommendations: the immediate fix and how to prevent a repeat.

SUMMARY

SPLIT	INCIDENTS	RECOVERED	DIAGNOSIS F1	MEAN SCORE	COLLATERAL
dev	10	100%	0.97	99.0	0

MEND

INCIDENTS

INCIDENT	SPLIT	RESULT	SCORE	REPORTED ROOT CAUSE
dev-01-api-down	dev	RECOVERED	100	api/service_down
dev-02-postgres-down	dev	RECOVERED	100	postgres/service_down
dev-03-redis-down	dev	RECOVERED	100	redis/service_down
dev-04-db-password	dev	RECOVERED	100	api/bad_config_deploy
dev-05-gateway-upstream	dev	RECOVERED	100	gateway/bad_config_deploy
dev-06-disk-full	dev	RECOVERED	100	api/disk_full
dev-07-cpu-hog	dev	RECOVERED	100	api/cpu_exhaustion
dev-08-postgres-partition	dev	RECOVERED	100	postgres/network_partition
dev-09-multi-disk-password	dev	RECOVERED	90	api/bad_config_deploy
dev-10-multi-redis-gateway	dev	RECOVERED	100	redis/service_down; gateway/bad_config_deploy

DEV

dev-01-api-down

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2003ms over the last few seconds.
  - GET /api/health: 1/1 failed, statuses [504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - GET /api/products: 1/1 failed, statuses [504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - GET /api/orders/recent: 1/1 failed, statuses [504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - POST /api/orders: 1/1 failed, statuses [504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`service_stopped` on `api`: A core service process is stopped.
Root causes to name: `api/service_down`

MEND AI'S INCIDENT REPORT

`api/service_down`: api container is exited (list_services); 4 endpoint(s) failing while api is down
Summary: [confirmed] start the api container (confidence 0.93; evidence: api container is exited (list_services)).
Prevent: add a restart policy and alert on container exits.

INVESTIGATION AND ACTIONS · 26 TOOL CALLS · 1 STATE CHANGES · 31.3 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	8.2s	OBSERVE	`get_recent_changes(limit=20)`
4	8.2s	OBSERVE	`get_metrics()`
5	10.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	10.4s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	10.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	10.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	10.5s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	10.6s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
11	10.7s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`

#	TIME	TYPE	TOOL CALL
12	10.8s	ACT	`start_service(service=api)`
13	19.6s	OBSERVE	`list_services()`
14	19.7s	OBSERVE	`check_endpoints()`
15	19.8s	OBSERVE	`get_recent_changes(limit=20)`
16	19.8s	OBSERVE	`get_metrics()`
17	22.1s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
18	22.1s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
19	22.2s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
20	22.2s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
21	22.2s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
22	22.3s	OBSERVE	`run_diagnostic(service=api, check=disk)`
23	22.4s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
24	22.4s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
25	22.6s	OBSERVE	`verify_recovery()`
26	31.3s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'service_down', 'evidence': 'api container is exited (list_services); 4 endpoint(s) failing while api is down'}], recovered=True, summary=[confirmed] start the api container (confidence 0.93; evidence: api container is exited (list_services)). Prevent: add a restart policy and alert on container exits.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Start the api service (start_service), then confirm the endpoints recover.

Prevent a repeat

- Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
- Alert on service state and container exits, not only on failing user traffic.
- Record who stops production services and require a change ticket for manual stops.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-02-postgres-down

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 6ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2]
Name does not resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`service_stopped` on `postgres`: A core service process is stopped.
Root causes to name: `postgres/service_down`

MEND AI'S INCIDENT REPORT

`postgres/service_down`: postgres container is exited (list_services); 3 endpoint(s) failing while postgres is down

Summary: [confirmed] start the postgres container (confidence 0.93; evidence: postgres container is exited (list_services)). Prevent: add a restart policy and alert on container exits.

INVESTIGATION AND ACTIONS · 26 TOOL CALLS · 1 STATE CHANGES · 22.6 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.3s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.4s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.4s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.4s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.4s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.5s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.5s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.6s	ACT	`start_service(service=postgres)`
13	10.9s	OBSERVE	`list_services()`
14	11.1s	OBSERVE	`check_endpoints()`
15	11.1s	OBSERVE	`get_recent_changes(limit=20)`

MEND

#	TIME	TYPE	TOOL CALL
16	11.1s	OBSERVE	`get_metrics()`
17	13.4s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
18	13.4s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
19	13.4s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
20	13.5s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
21	13.5s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
22	13.5s	OBSERVE	`run_diagnostic(service=api, check=disk)`
23	13.6s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
24	13.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
25	13.9s	OBSERVE	`verify_recovery()`
26	22.6s	FINAL	`submit_report(root_causes=[{'component': 'postgres', 'category': 'service_down', 'evidence': 'postgres container is exited (list_services); 3 endpoint(s) failing while postgres is down'}], recovered=True, summary=[confirmed] start the postgres container (confidence 0.93; evidence: postgres container is exited (list_services)). Prevent: add a restart policy and alert on container exits.)`

INDEPENDENT VERIFICATION

100% user traffic success	**24 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Start the postgres service (start_service), then confirm the endpoints recover.

Prevent a repeat

- Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
- Alert on service state and container exits, not only on failing user traffic.
- Record who stops production services and require a change ticket for manual stops.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-03-redis-down

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 22ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to
redis:6379. Name does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`service_stopped` on `redis`: A core service process is stopped.
Root causes to name: `redis/service_down`

MEND AI'S INCIDENT REPORT

`redis/service_down`: redis container is exited (list_services); 1 endpoint(s) failing while redis is down
Summary: [confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)).
Prevent: add a restart policy and alert on container exits.

INVESTIGATION AND ACTIONS · 27 TOOL CALLS · 1 STATE CHANGES · 22.9 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.6s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
13	2.9s	ACT	`start_service(service=redis)`
14	11.2s	OBSERVE	`list_services()`
15	11.4s	OBSERVE	`check_endpoints()`
16	11.4s	OBSERVE	`get_recent_changes(limit=20)`
17	11.4s	OBSERVE	`get_metrics()`
18	13.7s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`

MEND

#	TIME	TYPE	TOOL CALL
19	13.8s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
20	13.8s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
21	13.8s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
22	13.8s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
23	13.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
24	14.0s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
25	14.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
26	14.2s	OBSERVE	`verify_recovery()`
27	22.9s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'service_down', 'evidence': 'redis container is exited (list_services); 1 endpoint(s) failing while redis is down'}], recovered=True, summary=[confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)). Prevent: add a restart policy and alert on container exits.)`

INDEPENDENT VERIFICATION

100% user traffic success	**24 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Start the redis service (start_service), then confirm the endpoints recover.

Prevent a repeat

- Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
- Alert on service state and container exits, not only on failing user traffic.
- Record who stops production services and require a change ticket for manual stops.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-04-db-password

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 16ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection
failed: connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`bad_db_password` on `api`: Config deploy with wrong DB password.
Root causes to name: `api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:10:44 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'rotate database credentials (SEC-4411)' by ci-bot

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:10:44 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`

#	TIME	TYPE	TOOL CALL
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	2.9s	ACT	`rollback_config(service=api, to_version=1)`
16	21.5s	OBSERVE	`list_services()`
17	21.6s	OBSERVE	`check_endpoints()`
18	21.7s	OBSERVE	`get_recent_changes(limit=20)`
19	21.7s	OBSERVE	`get_metrics()`
20	24.0s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	24.0s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	24.0s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	24.0s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.1s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.3s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.4s	OBSERVE	`verify_recovery()`
29	33.1s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '19 log line(s) on api: the database rejected the client\'s credentials, e.g. 2026-10-30T03:10:44 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 \'rotate database credentials (SEC-4411)\' by ci-bot'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:10:44 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**20 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version with the working database password.

Prevent a repeat

- Rotate credentials through a secrets manager in step with the database, never by editing app config.
- Run a pre-deploy check that the new config can authenticate to the database.
- Deploy config as a canary and roll back automatically when health checks fail.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-05-gateway-upstream

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 1ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`gateway_bad_upstream` on `gateway`: Gateway config deploy with wrong upstream port.

Red herrings planted: `benign_api_deploy`

Root causes to name: `gateway/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`gateway/bad_config_deploy`: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:12:34 [error] 26#26: *587 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors

Summary: [confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:12:34 [error] 26#26: *587 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 30 TOOL CALLS · 1 STATE CHANGES · 22.7 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`

#	TIME	TYPE	TOOL CALL
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.5s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=gateway, check=upstream_http)`
11	2.6s	OBSERVE	`get_config(service=gateway)`
12	2.6s	OBSERVE	`get_config(service=gateway, version=2, diff_against=1)`
13	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
14	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
15	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
16	2.9s	ACT	`rollback_config(service=gateway, to_version=1)`
17	11.0s	OBSERVE	`list_services()`
18	11.2s	OBSERVE	`check_endpoints()`
19	11.2s	OBSERVE	`get_recent_changes(limit=20)`
20	11.2s	OBSERVE	`get_metrics()`
21	13.5s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
22	13.6s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
23	13.6s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
24	13.6s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
25	13.7s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
26	13.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
27	13.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
28	13.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
29	14.0s	OBSERVE	`verify_recovery()`
30	22.7s	FINAL	`submit_report(root_causes=[{'component': 'gateway', 'category': 'bad_config_deploy', 'evidence': '12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:12:34 [error] 26#26: *587 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors'}], recovered=True, summary=[confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:12:34 [error] 26#26: *587 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**20 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the gateway config to the last version with the correct api upstream port (api:8000).

Prevent a repeat

- Run nginx -t and an upstream connectivity check as part of every gateway deploy.
- Keep upstream addresses in one shared service registry instead of hand-edited configs.
- Deploy gateway changes with automatic rollback when 5xx rates jump.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-06-disk-full

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 21ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"ok","database":"ok","disk":"error:
[Errno 28] No space left )
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`disk_full` on `api`: A runaway export fills the API data volume.
Root causes to name: `api/disk_full`

MEND AI'S INCIDENT REPORT

`api/disk_full`: api /data volume is 100% full (run_diagnostic disk); 1 'No space left on device' errors, e.g. probe /api/health: {"checks":{"cache":"ok","database":"ok","disk":"error: [Errno 28] No space left on device"},"status"

Summary: [confirmed] delete disposable temp files on api: /data/tmp/export-orders-20261029.csv.tmp (confidence 0.82; evidence: api /data volume is 100% full (run_diagnostic disk)). Prevent: rotate or expire temporary exports and alert on volume usage above 80%.

INVESTIGATION AND ACTIONS · 27 TOOL CALLS · 1 STATE CHANGES · 19.6 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.2s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.6s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.6s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.7s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
13	2.9s	ACT	`delete_file(service=api, path=/data/tmp/export-orders-20261029.csv.tmp)`
14	8.0s	OBSERVE	`list_services()`
15	8.2s	OBSERVE	`check_endpoints()`
16	8.2s	OBSERVE	`get_recent_changes(limit=20)`

#	TIME	TYPE	TOOL CALL
17	8.2s	OBSERVE	`get_metrics()`
18	10.5s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
19	10.5s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
20	10.5s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
21	10.5s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
22	10.6s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
23	10.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
24	10.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
25	10.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
26	10.9s	OBSERVE	`verify_recovery()`
27	19.6s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'disk_full', 'evidence': 'api /data volume is 100% full (run_diagnostic disk); 1 \'No space left on device\' errors, e.g. probe /api/health: {"checks":{"cache":"ok","database":"ok","disk":"error: [Errno 28] No space left on device"},"status"'}], recovered=True, summary=[confirmed] delete disposable temp files on api: /data/tmp/export-orders-20261029.csv.tmp (confidence 0.82; evidence: api /data volume is 100% full (run_diagnostic disk)). Prevent: rotate or expire temporary exports and alert on volume usage above 80%.)`

INDEPENDENT VERIFICATION

100% user traffic success	**20 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Delete the runaway temporary export under /data/tmp/ on api, then confirm /data is below 80%.

Prevent a repeat

- Alert when the data volume passes 80% so it is caught before writes fail.
- Write exports to object storage or a separate volume with a quota, not to the app's data volume.
- Clean up temporary files automatically (TTL) and cap export sizes.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-07-cpu-hog

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 696ms over the last few seconds.
  - all probes passing but p95 latency is 696ms
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`cpu_hog` on `api`: A runaway search reindex job saturates the API's CPU.
Root causes to name: `api/cpu_exhaustion`

MEND AI'S INCIDENT REPORT

`api/cpu_exhaustion`: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes); api container CPU at 52% (get_metrics); latency degraded: /api/orders/recent 596ms

Summary: [confirmed] stop the runaway 'python /app/tools/reindex_search.py' job on api (pid 23 and its children) (confidence 0.76; evidence: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes)). Prevent: run batch jobs outside the serving container or under a CPU quota.

INVESTIGATION AND ACTIONS · 28 TOOL CALLS · 1 STATE CHANGES · 23.7 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.2s	OBSERVE	`check_endpoints()`
3	1.6s	OBSERVE	`get_recent_changes(limit=20)`
4	1.6s	OBSERVE	`get_metrics()`
5	4.8s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	4.8s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	4.9s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	4.9s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	4.9s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	5.0s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	6.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	6.3s	OBSERVE	`run_diagnostic(service=api, check=processes)`
13	7.2s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
14	7.3s	ACT	`kill_process(service=api, pid=23)`
15	13.1s	OBSERVE	`list_services()`
16	13.2s	OBSERVE	`check_endpoints()`

#	TIME	TYPE	TOOL CALL
17	13.3s	OBSERVE	`get_recent_changes(limit=20)`
18	13.3s	OBSERVE	`get_metrics()`
19	14.6s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
20	14.6s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
21	14.6s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
22	14.6s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
23	14.7s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
24	14.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
25	14.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
26	14.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
27	15.0s	OBSERVE	`verify_recovery()`
28	23.7s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'cpu_exhaustion', 'evidence': "9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes); api container CPU at 52% (get_metrics); latency degraded: /api/orders/recent 596ms"}], recovered=True, summary=[confirmed] stop the runaway 'python /app/tools/reindex_search.py' job on api (pid 23 and its children) (confidence 0.76; evidence: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes)). Prevent: run batch jobs outside the serving container or under a CPU quota.)`

INDEPENDENT VERIFICATION

100% user traffic success	**21 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Stop the runaway reindex_search job on api (kill_process), then confirm latency returns to normal.

Prevent a repeat

- Run batch jobs like search reindexing on a worker or a schedule, never inside the serving api container.
- Set CPU limits per container so one job cannot starve request handling.
- Alert on sustained high CPU together with rising p95 latency.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-08-postgres-partition

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 5ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2]
Name does not resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`network_partition` on `postgres`: A backend service is cut off from the backend network.
Root causes to name: `postgres/network_partition`

MEND AI'S INCIDENT REPORT

`postgres/network_partition`: postgres is not attached to the backend network it belongs on (list_services); 35 name-resolution errors in logs, e.g. 2026-10-30T03:17:01 ERROR worker summary refresh failed: [Errno -2] Name does not resolve

Summary: [confirmed] re-attach postgres to the backend network (restores its DNS name) (confidence 0.94; evidence: postgres is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services.

INVESTIGATION AND ACTIONS · 27 TOOL CALLS · 1 STATE CHANGES · 22.7 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.6s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
13	3.0s	ACT	`reconnect_network(service=postgres, network=backend)`
14	11.1s	OBSERVE	`list_services()`

#	TIME	TYPE	TOOL CALL
15	11.2s	OBSERVE	`check_endpoints()`
16	11.3s	OBSERVE	`get_recent_changes(limit=20)`
17	11.3s	OBSERVE	`get_metrics()`
18	13.6s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
19	13.6s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
20	13.6s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
21	13.7s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
22	13.7s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
23	13.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
24	13.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
25	13.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
26	13.9s	OBSERVE	`verify_recovery()`
27	22.7s	FINAL	`submit_report(root_causes=[{'component': 'postgres', 'category': 'network_partition', 'evidence': 'postgres is not attached to the backend network it belongs on (list_services); 35 name-resolution errors in logs, e.g. 2026-10-30T03:17:01 ERROR worker summary refresh failed: [Errno -2] Name does not resolve'}], recovered=True, summary=[confirmed] re-attach postgres to the backend network (restores its DNS name) (confidence 0.94; evidence: postgres is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services.)`

INDEPENDENT VERIFICATION

100% user traffic success	**24 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Reconnect postgres to the backend network (reconnect_network), then confirm its dependents recover.

Prevent a repeat

- Manage network attachments only through docker compose or infrastructure code, never by hand.
- Add connectivity checks between dependent services to monitoring.
- Alert on dependency health (database/cache unreachable) with the exact component named.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-09-multi-disk-password

SYSTEM RECOVERED

90 / 100

recovery 60 · diagnosis 20 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 36%, p95 16ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection
failed: connection to serv)
  - GET /api/products: 1/5 failed, statuses [200, 503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`disk_full` on `api`: A runaway export fills the API data volume.
`bad_db_password` on `api`: Config deploy with wrong DB password.
Root causes to name: `api/disk_full; api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:18:45 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.2", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'rotate database credentials (SEC-4411)' by ci-bot

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:18:45 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.2", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`

#	TIME	TYPE	TOOL CALL
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	2.9s	ACT	`rollback_config(service=api, to_version=1)`
16	21.4s	OBSERVE	`list_services()`
17	21.6s	OBSERVE	`check_endpoints()`
18	21.6s	OBSERVE	`get_recent_changes(limit=20)`
19	21.6s	OBSERVE	`get_metrics()`
20	23.9s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	23.9s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	24.0s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	24.0s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.0s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.1s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.2s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.3s	OBSERVE	`verify_recovery()`
29	33.1s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '19 log line(s) on api: the database rejected the client\'s credentials, e.g. 2026-10-30T03:18:45 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.2", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 \'rotate database credentials (SEC-4411)\' by ci-bot'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:18:45 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.2", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**23 ms** p95 latency	**0** collateral actions	**0.67** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Delete the runaway temporary export under /data/tmp/ on api, then confirm /data is below 80%.
- Roll back the api config to the last version with the working database password.

Prevent a repeat

- Alert when the data volume passes 80% so it is caught before writes fail.
- Write exports to object storage or a separate volume with a quota, not to the app's data volume.
- Clean up temporary files automatically (TTL) and cap export sizes.
- Rotate credentials through a secrets manager in step with the database, never by editing app config.
- Run a pre-deploy check that the new config can authenticate to the database.
- Deploy config as a canary and roll back automatically when health checks fail.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

DEV

dev-10-multi-redis-gateway

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`service_stopped` on `redis`: A core service process is stopped.
`gateway_bad_upstream` on `gateway`: Gateway config deploy with wrong upstream port.
Red herrings planted: `worker_warnings`
Root causes to name: `redis/service_down; gateway/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`redis/service_down`: redis container is exited (list_services); 4 endpoint(s) failing while redis is down
`gateway/bad_config_deploy`: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:20:38 [error] 27#27: *654 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors
Summary: [confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)). Prevent: add a restart policy and alert on container exits. I [confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:20:38 [error] 27#27: *654 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 46 TOOL CALLS · 2 STATE CHANGES · 34.3 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`

#	TIME	TYPE	TOOL CALL
6	2.5s	OBSERVE	get_logs(service=api, since_seconds=120, tail=400)
7	2.5s	OBSERVE	get_logs(service=worker, since_seconds=120, tail=400)
8	2.5s	OBSERVE	get_logs(service=postgres, since_seconds=120, tail=400)
9	2.5s	OBSERVE	get_logs(service=redis, since_seconds=120, tail=400)
10	2.6s	OBSERVE	run_diagnostic(service=gateway, check=upstream_http)
11	2.7s	OBSERVE	get_config(service=gateway)
12	2.7s	OBSERVE	get_config(service=gateway, version=2, diff_against=1)
13	2.7s	OBSERVE	run_diagnostic(service=api, check=disk)
14	2.8s	OBSERVE	run_diagnostic(service=worker, check=disk)
15	2.9s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
16	3.0s	ACT	start_service(service=redis)
17	11.3s	OBSERVE	list_services()
18	11.5s	OBSERVE	check_endpoints()
19	11.5s	OBSERVE	get_recent_changes(limit=20)
20	11.5s	OBSERVE	get_metrics()
21	13.8s	OBSERVE	get_logs(service=gateway, since_seconds=9, tail=400)
22	13.8s	OBSERVE	get_logs(service=api, since_seconds=9, tail=400)
23	13.8s	OBSERVE	get_logs(service=worker, since_seconds=9, tail=400)
24	13.9s	OBSERVE	get_logs(service=postgres, since_seconds=9, tail=400)
25	13.9s	OBSERVE	get_logs(service=redis, since_seconds=9, tail=400)
26	13.9s	OBSERVE	run_diagnostic(service=gateway, check=upstream_http)
27	14.1s	OBSERVE	get_config(service=gateway)
28	14.1s	OBSERVE	get_config(service=gateway, version=2, diff_against=1)
29	14.1s	OBSERVE	run_diagnostic(service=api, check=disk)
30	14.2s	OBSERVE	run_diagnostic(service=worker, check=disk)
31	14.3s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
32	14.4s	ACT	rollback_config(service=gateway, to_version=1)
33	22.6s	OBSERVE	list_services()
34	22.7s	OBSERVE	check_endpoints()
35	22.7s	OBSERVE	get_recent_changes(limit=20)
36	22.7s	OBSERVE	get_metrics()
37	25.1s	OBSERVE	get_logs(service=gateway, since_seconds=9, tail=400)
38	25.1s	OBSERVE	get_logs(service=api, since_seconds=9, tail=400)
39	25.1s	OBSERVE	get_logs(service=worker, since_seconds=9, tail=400)

#	TIME	TYPE	TOOL CALL
40	25.1s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
41	25.2s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
42	25.2s	OBSERVE	`run_diagnostic(service=api, check=disk)`
43	25.3s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
44	25.4s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
45	25.5s	OBSERVE	`verify_recovery()`
46	34.3s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'service_down', 'evidence': 'redis container is exited (list_services); 4 endpoint(s) failing while redis is down'}, {'component': 'gateway', 'category': 'bad_config_deploy', 'evidence': '12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:20:38 [error] 27#27: *654 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors'}], recovered=True, summary=[confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)). Prevent: add a restart policy and alert on container exits. | [confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:20:38 [error] 27#27: *654 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Start the redis service (start_service), then confirm the endpoints recover.
- Roll back the gateway config to the last version with the correct api upstream port (api:8000).

Prevent a repeat

- Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
- Alert on service state and container exits, not only on failing user traffic.
- Record who stops production services and require a change ticket for manual stops.
- Run nginx -t and an upstream connectivity check as part of every gateway deploy.
- Keep upstream addresses in one shared service registry instead of hand-edited configs.
- Deploy gateway changes with automatic rollback when 5xx rates jump.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.